

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 5 Procesamiento y Optimización de Consultas

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Ejercicio 1: Algoritmos de selección	2
2.1. Marco teórico aplicado (Teórico 6)	3
2.2. Resolución por inciso	3
2.3. Resumen	4
3. Ejercicio 2: Árbol de query y expresiones equivalentes	4
3.1. Marco teórico aplicado (Teórico 7)	5
3.2. a) $\sigma_{desc="Pan Rallado" \wedge precio_venta < 15}(proveedor \bowtie provee \bowtie articulos)$	5
3.3. b) $\sigma_{precio > 25}(cliente \bowtie compran \bowtie articulos)$	6
4. Ejercicio 3: Estimación de tamaño y orden de join	7
4.1. Marco teórico aplicado (Teórico 7)	7
4.2. Estimación de cardinalidad	8
4.3. Orden de ejecución más eficiente	8
5. Ejercicio 4: Catálogo de PostgreSQL	9
5.1. a) Metadata de tablas, atributos y tipos	9
5.2. b) Estadísticas que usa el planner	9
6. Ejercicio 5: Ejecución y planes en PostgreSQL	11
6.1. a) Creación de tablas y carga inicial	11
6.2. b) Programa que inserta una fila por par persona-recurso	11
6.3. c) Información estadística del catálogo	12
6.4. d) Filtrado por fecha: plan inicial y mejora	13
6.5. e) NATURAL JOIN: análisis y forzado de Merge Join	14
7. Conclusiones	15

1. Introducción

El Práctico 5 cierra la parte del cuatrimestre dedicada a entender qué hace un motor de base de datos por dentro cuando recibe una consulta. Los cinco ejercicios cubren los tres bloques en los que se descompone el procesamiento de un *query*:

- **Procesamiento físico:** qué algoritmo concreto se usa para cada operación del álgebra relacional (Ejercicio 1).
- **Optimización lógica:** cómo se reescribe el árbol de la consulta usando reglas de equivalencia para bajar selecciones, ordenar joins y reducir tamaños intermedios (Ejercicios 2 y 3).
- **Catálogo y estadísticas:** dónde se guarda en PostgreSQL la información que el *planner* usa para decidir el plan, y cómo se influye sobre el plan elegido (Ejercicios 4 y 5).

La resolución se apoya en los Teóricos 6 (*Procesamiento de Consultas*, algoritmos A1–A11, costo en bloques transferidos y búsquedas) y 7 (*Optimización de Consultas*, reglas de equivalencia, optimización por costos y heurística).

Notación. A lo largo del documento se usa la notación clásica del álgebra relacional: σ para selección, Π para proyección y $\bowtie$ para join (natural cuando no se aclara la condición). Para los costos se siguen las variables del Teórico 6: t_T tiempo de transferencia de un bloque, t_b tiempo de búsqueda, b_r bloques de la relación r , n_r tuplas de r , h_i altura del índice.

Verificación de las resoluciones previas

Antes de redactar este documento se revisaron los README de cada ejercicio y los scripts SQL del Ejercicio 5. Conclusiones:

- **Ejercicio 1:** la versión original eligió **A1** para los incisos (a) y (b) interpretando que sólo existían los índices secundarios listados en el enunciado. Se corrigió a **A3** (igualdad sobre índice primario/único) y **A6** (rango sobre índice primario), que son las respuestas más rigurosas: una clave primaria, en cualquier implementación realista y en la convención del curso, tiene asociado un índice único. Los incisos (c)–(e) ya estaban correctos (A5, A7, A10).
- **Ejercicio 2:** las tres transformaciones por inciso son correctas (cascada de selecciones, push-down y asociatividad/conmutatividad de join). Se completaron árboles de expresión antes y después.
- **Ejercicio 3:** estimación y orden estaban bien. Se agregó la cota formal $|R \bowtie S| \leq \min(n_r, n_s)$ cuando el atributo común es clave, y la comparación cuantitativa de los dos órdenes posibles.
- **Ejercicio 4:** las consultas a `information_schema` y `pg_catalog` son correctas. Se reorganizó la información sobre qué estadísticas guarda `pg_stats` con explicación de su uso por el planner.
- **Ejercicio 5:** los scripts SQL (carga base, producto cartesiano, catálogo, mejora con índice y fuerza de *merge join*) funcionan tal como están. Se sumó interpretación de los planes y la justificación del cambio observado.

2. Ejercicio 1: Algoritmos de selección

Enunciado. Sobre la tabla

`curso(codigo, nombre, cantidad_inscriptos, cupo_maximo)`

con índice secundario sobre `cantidad_inscriptos` y sobre `cupo_maximo`, especificar qué algoritmo (A1–A11) se usaría para cada una de las consultas (a)–(e) y justificar.

2.1. Marco teórico aplicado (Teórico 6)

Los algoritmos relevantes son:

- **A1:** búsqueda lineal. Aplicable en cualquier caso. Costo $b_r \cdot t_T + t_b$; si la condición es por clave y hay igualdad, se corta apenas se encuentra: $(b_r/2) \cdot t_T + t_b$.
- **A2:** búsqueda binaria sobre archivo ordenado.
- **A3:** igualdad sobre índice de clave primaria o único. Recupera a lo sumo una tupla. Costo $(h_i + 1)(t_T + t_b)$.
- **A4:** igualdad sobre índice primario no único.
- **A5:** igualdad sobre índice secundario. Si la clave no es única puede devolver varios registros, cada uno en un bloque distinto.
- **A6:** rango sobre índice primario (recorre secuencialmente).
- **A7:** rango sobre índice secundario. Para cada puntero, una operación de I/O.
- **A8:** conjunción usando *un* índice + chequeo en memoria.
- **A9:** conjunción usando un índice multiclave.
- **A10:** conjunción por intersección de identificadores (RIDs/punteros).
- **A11:** disyunción por unión de identificadores.

2.2. Resolución por inciso

a) $\sigma_{\text{codigo}=1001}(\text{curso})$

Algoritmo: **A3** (igualdad sobre índice primario/único).

Justificación. `codigo` es clave primaria; por convención del curso (y por implementación real) toda PK lleva asociado un índice único, casi siempre un B+tree. La condición es igualdad sobre ese atributo. A3 recupera a lo sumo una tupla y su costo, comparado con un *scan* completo, es muy bajo:

$$\text{Costo}_{A3} = (h_i + 1)(t_T + t_b)$$

Interpretación estrictamente literal del enunciado. Si se asumiera que sólo existen los índices que el enunciado menciona (los dos secundarios), no habría índice sobre `codigo`: el algoritmo aplicable sería **A1** con corte temprano por ser clave: $(b_r/2)t_T + t_b$. A3 es la respuesta más rigurosa.

b) $\sigma_{\text{codigo}<1000}(\text{curso})$

Algoritmo: **A6** (rango sobre índice primario).

Justificación. Misma convención que en (a): la PK tiene índice asociado, y si se interpreta como índice primario (archivo organizado por `codigo`), se recorren las tuplas en orden desde el inicio hasta encontrar la primera que viole la condición.

Si se interpreta la PK sólo como restricción de unicidad sin índice primario explícito, la respuesta cae en **A1** (búsqueda lineal completa). En cualquier caso, una solución con índice (B+tree) es preferible a A_1 siempre que la selectividad no sea muy alta.

c) $\sigma_{cantidad_inscriptos=40}(\text{curso})$

Algoritmo: **A5** (igualdad sobre índice secundario).

Justificación. Hay índice secundario sobre `cantidad_inscriptos`; la condición es de igualdad. Como `cantidad_inscriptos` no es clave, pueden satisfacer la condición varios registros, posiblemente uno en cada bloque. Costo del peor caso:

$$\text{Costo}_{A5} = (h_i + n)(t_T + t_b)$$

donde n es la cantidad de tuplas que cumplen la condición.

d) $\sigma_{cantidad_inscriptos < 50}(\text{curso})$

Algoritmo: **A7** (rango sobre índice secundario).

Justificación. Hay índice secundario sobre el atributo y la condición es de rango. A7 recorre las hojas del índice secundario para conseguir los punteros que cumplen `cantidad_inscriptos < 50` y luego, por cada puntero, hace una I/O para recuperar la tupla.

Observación práctica. A7 es atractivo cuando la selectividad es baja (pocos registros cumplen la condición). Si la mayoría de la tabla cumple `< 50`, un *Seq Scan* (A1) puede ser preferible porque evita las operaciones de I/O saltadas que produce el índice secundario.

e) $\sigma_{cantidad_inscriptos=50 \wedge cupo_maximo > 40}(\text{curso})$

Algoritmo: **A10** (conjunción por intersección de identificadores).

Justificación. La consulta es una conjunción de dos condiciones, ambas con índice secundario disponible:

- `cantidad_inscriptos = 50` se resuelve con el índice de `cantidad_inscriptos` (lógica de A5) y devuelve un conjunto de punteros P_1 .
- `cupo_maximo > 40` se resuelve con el índice de `cupo_maximo` (lógica de A7) y devuelve un conjunto P_2 .
- Se calcula $P_1 \cap P_2$ y se recuperan sólo las tuplas referidas por los punteros que sobreviven.

Alternativas. También sería válido A8 (elegir la condición más selectiva, recuperar por su índice y testear el resto en memoria). Si la condición más selectiva es la igualdad sobre `cantidad_inscriptos`, A8 con A5 puede ser una opción razonable y simple de implementar.

2.3. Resumen

Inciso	Condición	Algoritmo
a	<code>codigo = 1001</code>	A3
b	<code>codigo < 1000</code>	A6
c	<code>cantidad_inscriptos = 40</code>	A5
d	<code>cantidad_inscriptos < 50</code>	A7
e	<code>cantidad_inscriptos = 50 ∧ cupo_maximo > 40</code>	A10

3. Ejercicio 2: Árbol de query y expresiones equivalentes

Enunciado. Sobre el esquema

- `articulos(Nart, desc, precio, cant, stock_min, stock_max)`
- `provee(Nprov, Nart, precio_venta)`
- `proveedor(Nprov, nombre, direccion)`
- `cliente(Ncli, nombre, direccion)`
- `compran(Ncli, Nart)`

obtener, para cada consulta, el árbol de query inicial y *tres* expresiones equivalentes, justificando cada paso.

3.1. Marco teórico aplicado (Teórico 7)

Las transformaciones se apoyan en las **reglas de equivalencia** del Teórico 7:

- **R1** (cascada de selecciones): $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$.
- **R2** (conmutatividad de selecciones): $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$.
- **R5** (conmutatividad de join): $E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$.
- **R6a** (asociatividad de join natural): $(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$.
- **R7a** (push-down de selección dentro de join, condición sobre un solo operando): $\sigma_{\theta_0}(E_1 \bowtie E_2) \equiv \sigma_{\theta_0}(E_1) \bowtie E_2$ si θ_0 involucra sólo atributos de E_1 .
- **R7b** (push-down distributivo cuando la condición es conjuntiva con partes que pertenecen a operandos distintos): $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie E_2) \equiv \sigma_{\theta_1}(E_1) \bowtie \sigma_{\theta_2}(E_2)$ si θ_1 sólo refiere E_1 y θ_2 sólo a E_2 .

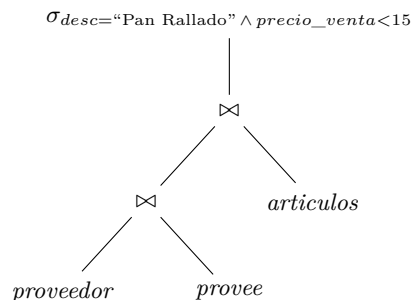
La heurística principal del curso es *empujar selecciones cerca de las hojas* (achican la cardinalidad), *empujar proyecciones* (achican el ancho de tupla) y *ordenar los joins* para minimizar el tamaño de los resultados intermedios.

3.2. a) $\sigma_{desc=\text{"Pan Rallado"}} \wedge precio_venta < 15 (proveedor \bowtie provee \bowtie articulos)$

Expresión base E_0

$$E_0 = \sigma_{desc=\text{"Pan Rallado"}} \wedge precio_venta < 15 (proveedor \bowtie provee \bowtie articulos)$$

Árbol de E_0 .



Equivalente E_1 — cascada de selecciones (R1)

$$E_1 = \sigma_{desc=\text{"Pan Rallado"}}(\sigma_{precio_venta < 15}((proveedor \bowtie provee) \bowtie articulos))$$

Se reemplazó una selección conjuntiva por dos selecciones encadenadas. Esto no mejora costo todavía pero habilita el paso siguiente: ahora cada σ involucra un sólo predicado y puede empujarse de manera independiente.

Equivalente E_2 — push-down por R7b

Análisis de atributos:

- `precio_venta` sólo está en `provee`.
- `desc` sólo está en `articulos`.

Aplicando R7a/R7b se distribuye cada selección a la rama del join donde aparecen sus atributos:

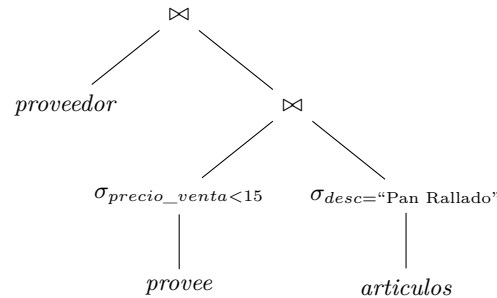
$$E_2 = (\text{proveedor} \bowtie \sigma_{\text{precio_venta} < 15}(\text{provee})) \bowtie \sigma_{\text{desc} = \text{"Pan Rallado"}}(\text{articulos})$$

Equivalente E_3 — reasociación (R5, R6a)

Conviene calcular primero el join entre las dos relaciones *ya filtradas* (que son las más chicas):

$$E_3 = \text{proveedor} \bowtie (\sigma_{\text{precio_venta} < 15}(\text{provee}) \bowtie \sigma_{\text{desc} = \text{"Pan Rallado"}}(\text{articulos}))$$

Árbol final (E_3).



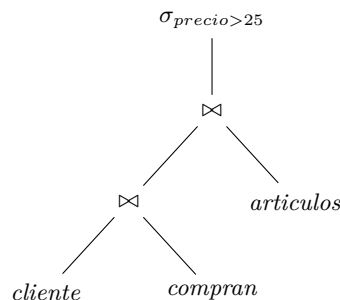
Por qué este árbol es mejor. Las dos selecciones se evalúan sobre tablas base (no sobre productos intermedios), y reducen la cardinalidad antes del primer join. El primer $\bowtie$ une los resultados filtrados (típicamente mucho más pequeños), por lo que la tabla intermedia de cara al segundo join es chica.

3.3. b) $\sigma_{\text{precio} > 25}(\text{cliente} \bowtie \text{compran} \bowtie \text{articulos})$

Expresión base E_0

$$E_0 = \sigma_{\text{precio} > 25}((\text{cliente} \bowtie \text{compran}) \bowtie \text{articulos})$$

Árbol de E_0 .



Equivalente E_1 — push-down a artículos (R7a)

precio sólo aparece en *articulos*, por lo que se empuja la selección a esa hoja:

$$E_1 = (\text{cliente} \bowtie \text{compran}) \bowtie \sigma_{\text{precio} > 25}(\text{articulos})$$

Equivalente E_2 — asociatividad (R6a)

$$E_2 = \text{cliente} \bowtie (\text{compran} \bowtie \sigma_{\text{precio} > 25}(\text{articulos}))$$

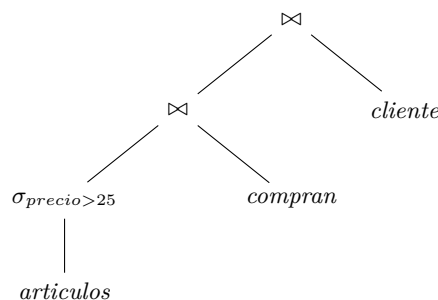
Conviene calcular primero $\text{compran} \bowtie \sigma_{\text{precio} > 25}(\text{articulos})$ porque σ ya redujo *articulos*; el resultado de ese join se cruza después con *cliente*.

Equivalente E_3 — conmutatividad y reasociación (R5, R6a)

$$E_3 = (\sigma_{\text{precio} > 25}(\text{articulos}) \bowtie \text{compran}) \bowtie \text{cliente}$$

Equivalente operacionalmente a E_2 pero útil si el algoritmo de join se implementa de manera asimétrica (por ejemplo, *hash join* con la relación más chica como *build*).

Árbol final (E_3).



Si se quisiera *también* proyectar tan pronto como sea posible, podría aplicarse R8 (proyección distribuye sobre join) para retener únicamente *Nart* en la rama de *articulos* filtrada y *Ncli* en *cliente*, achicando aún más las tuplas intermedias.

4. Ejercicio 3: Estimación de tamaño y orden de join

Enunciado. Relaciones $R_1(A, B, C)$, $R_2(C, D, E)$, $R_3(E, F)$ con claves A , C y E respectivamente, $|R_1| = 250$, $|R_2| = 3000$, $|R_3| = 5000$. Estimar $|R_1 \bowtie R_2 \bowtie R_3|$ y dar el orden de ejecución más eficiente.

4.1. Marco teórico aplicado (Teórico 7)

Cuando dos relaciones se unen por un atributo A que es *clave* en una de ellas, cada tupla de la otra se cruza con **a lo sumo una** tupla:

$$\text{Si } R \cap S = \{A\} \text{ y } A \text{ es clave de } S, \text{ entonces } |R \bowtie S| \leq |R|.$$

Si en cambio A no es clave en ninguna de las dos, la estimación clásica es

$$|R \bowtie S| \approx \frac{n_r n_s}{\max\{V(A, r), V(A, s)\}}.$$

La heurística para elegir el orden de un join multirrelación es: *primero los joins que reduzcan más el resultado intermedio*, dejando para el final las relaciones que aporten más tuplas al cómputo.

4.2. Estimación de cardinalidad

Paso 1: $R_1 \bowtie R_2$ (**atributo común C**). C es clave en R_2 , así que cada tupla de R_1 matchea con a lo sumo una de R_2 :

$$|R_1 \bowtie R_2| \leq |R_1| = 250.$$

Asumiendo integridad referencial usual (cada $R_1.C$ está presente en $R_2.C$), se cumple la igualdad: $|R_1 \bowtie R_2| = 250$.

Paso 2: $(R_1 \bowtie R_2) \bowtie R_3$ (**atributo común E**). E es clave en R_3 , por lo tanto cada tupla intermedia matchea con a lo sumo una de R_3 :

$$|R_1 \bowtie R_2 \bowtie R_3| \leq |R_1 \bowtie R_2| = 250.$$

Resultado. La cardinalidad estimada es $|R_1 \bowtie R_2 \bowtie R_3| = 250$ tuplas.

4.3. Orden de ejecución más eficiente

Hay dos planes razonables (ambos son válidos por asociatividad y conmutatividad del join natural):

Plan A: $(R_1 \bowtie R_2) \bowtie R_3$.

- Intermedio: $|R_1 \bowtie R_2| = 250$ tuplas.
- Join final: 250 tuplas contra R_3 (5000 tuplas) por E (clave en R_3).

Plan B: $R_1 \bowtie (R_2 \bowtie R_3)$.

- Intermedio: $|R_2 \bowtie R_3|$. Como E es clave en R_3 , $|R_2 \bowtie R_3| \leq |R_2| = 3000$. Bajo integridad referencial, $|R_2 \bowtie R_3| = 3000$.
- Join final: 3000 tuplas contra R_1 (250 tuplas) por C (clave en R_1).

Comparación

Plan	Tamaño intermedio	Tamaño final
A: $(R_1 \bowtie R_2) \bowtie R_3$	250	250
B: $R_1 \bowtie (R_2 \bowtie R_3)$	3000	250

El **Plan A** se queda con un resultado intermedio 12 veces más chico. Ese resultado se almacena (o se mantiene en pipeline) y se compara contra R_3 en el último join. Cualquier algoritmo concreto (*nested loop*, *hash join*, *merge join*) trabaja mejor con la relación externa más pequeña; el Plan A se la garantiza.

Conclusión.

Orden recomendado: $((R_1 \bowtie R_2) \bowtie R_3)$ con tamaño estimado 250.

5. Ejercicio 4: Catálogo de PostgreSQL

5.1. a) Metadata de tablas, atributos y tipos

PostgreSQL ofrece dos caminos para inspeccionar metadata:

Estándar `information_schema`.

- `information_schema.tables`: tablas y vistas accesibles.
- `information_schema.columns`: una fila por columna, con `column_name`, `data_type`, `udt_name`, longitud/precisión, nullability, default, posición, etc.

Catálogo interno (`pg_catalog`).

- `pg_class`: objetos relacionales (tablas, índices, secuencias, vistas).
- `pg_namespace`: esquemas.
- `pg_attribute`: columnas de cada relación (`attname`, `atttypid`, `attnum...`).
- `pg_type`: catálogo de tipos.

Consulta pedida: nombre y tipo de los atributos de vehiculo

Versión simple, sobre el estándar:

```
SELECT c.column_name AS atributo,
       c.data_type   AS tipo,
       c.udt_name    AS tipo_interno
FROM information_schema.columns c
WHERE c.table_schema = 'public'
      AND c.table_name = 'vehiculo'
ORDER BY c.ordinal_position;
```

Versión `pg_catalog` (equivalente, más cercana al catálogo real):

```
SELECT a.attname          AS atributo,
       format_type(a.atttypid, a.atttypmod) AS tipo
FROM pg_catalog.pg_attribute a
JOIN pg_catalog.pg_class    c  ON c.oid = a.attrelid
JOIN pg_catalog.pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'public'
      AND c.relname = 'vehiculo'
      AND a.attnum > 0           -- excluye columnas del sistema
      AND NOT a.attisdropped     -- excluye columnas borradas logicamente
ORDER BY a.attnum;
```

5.2. b) Estadísticas que usa el planner

Estadísticas de tabla

Las estimaciones del planner por tabla viven en `pg_class`:

- `reltuples`: estimación de la cantidad de filas (actualizada por `ANALYZE` y `VACUUM`).
- `relpages`: cantidad estimada de páginas (bloques).

Las estadísticas de actividad (cuántos scans secuenciales hubo, cuántos inserts, último `ANALYZE`, etc.) están en `pg_stat_user_tables`.

```

SELECT n.nspname      AS esquema,
       c.relname       AS tabla,
       c.reltuples     AS filas_estimadas,
       c.relpages      AS paginas
FROM pg_catalog.pg_class c
JOIN pg_catalog.pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
      AND c.relname = 'vehiculo';

```

Estadísticas por columna

La fuente real es la tabla interna `pg_statistic`; para consulta humana se usa la vista `pg_stats`, que decodifica los datos. `pg_stats` mantiene, por cada columna analizada:

- `null_frac`: fracción de valores NULL.
- `n_distinct`: cantidad de valores distintos (positivo, o negativo si se expresa como fracción de la tabla).
- `avg_width`: ancho promedio del valor en bytes.
- `most_common_vals` / `most_common_freqs`: valores más frecuentes y su frecuencia. Permite estimar selectividad para igualdades sobre valores *populares*.
- `histogram_bounds`: límites de un histograma equi-depth para estimar selectividad de rangos.
- `correlation`: correlación entre el orden lógico del atributo y el orden físico en disco. Cercano a 1 implica que el atributo es buen candidato para acceso por índice.

```

SELECT s.attnname,
       s.null_frac,
       s.n_distinct,
       s.avg_width,
       s.correlation,
       s.most_common_vals,
       s.most_common_freqs,
       s.histogram_bounds
FROM pg_catalog.pg_stats s
WHERE s.schemaname = 'public'
      AND s.tablename = 'vehiculo'
ORDER BY s.attnname;

```

Si la consulta a `pg_stats` no devuelve nada o devuelve datos desactualizados, hay que ejecutar `ANALYZE public.vehiculo;`. El recolector de estadísticas no actualiza estos números con cada modificación, sino que se dispara durante `VACUUM ANALYZE` o cuando `autovacuum` considera que se acumularon suficientes cambios.

Conexión con el Teórico 7

Estos datos son los n_r , $V(A, r)$, histogramas y MCVs que el Teórico 7 nombra al hablar de estimación de cardinalidades. El planner los usa para:

- estimar el tamaño de cada selección ($\sigma_{A=v}(r) \approx n_r/V(A, r)$),
- estimar el tamaño de joins ($n_r n_s / \max(V(A, r), V(A, s))$),
- decidir entre Seq Scan / Index Scan / Bitmap Heap Scan,
- elegir el algoritmo de join (*nested loop*, *hash*, *merge*).

6. Ejercicio 5: Ejecución y planes en PostgreSQL

Enunciado. Sobre el esquema

- `persona(persona_id, ...)` — script provisto por cátedra.
- `recurso(recurso_id, descripcion, ...)`.
- `seguimiento_acceso(seguimiento_acceso_id, fecha_yhora_entrada, fecha_yhora_salida, host, persona_id, recurso_id)`.

los cinco incisos piden: crear y poblar las tablas, generar accesos como producto cartesiano entre `persona` y `recurso`, observar el catálogo, y analizar los planes de dos consultas (filtrado por fecha y join natural), modificándolas para alterar la decisión del optimizador.

6.1. a) Creación de tablas y carga inicial

Los tres scripts provistos (`PERSONAPostgres.sql`, `RecursoPostgres.sql` y `Seguimiento_AceesoPostgres.sql`) se ejecutan en este orden. El tercero crea `seguimiento_acceso` con FK hacia `persona` y `recurso`, por eso va al final.

```
\i practico-05/ejercicio-05/sql/PERSONAPostgres.sql
\i practico-05/ejercicio-05/sql/RecursoPostgres.sql
\i practico-05/ejercicio-05/sql/Seguimiento_AceesoPostgres.sql

SELECT 'persona'           AS tabla, count(*) FROM persona
UNION ALL
SELECT 'recurso',          count(*) FROM recurso
UNION ALL
SELECT 'seguimiento_acceso', count(*) FROM seguimiento_acceso;
```

6.2. b) Programa que inserta una fila por par persona–recurso

La consigna pide “1 registro en `seguimiento_acceso` por cada persona y cada recurso”. La forma más directa en SQL puro es un `INSERT ...SELECT` con `CROSS JOIN`:

```
WITH pares AS (
    SELECT p.persona_id,
           r."RECURSO_ID" AS recurso_id,
           row_number() OVER (ORDER BY p.persona_id, r."RECURSO_ID") AS rn
    FROM persona p
    CROSS JOIN recurso r
),
faltantes AS (
    SELECT pa.*
    FROM pares pa
    WHERE NOT EXISTS (
        SELECT 1
        FROM seguimiento_acceso s
        WHERE s.persona_id = pa.persona_id
              AND s.recurso_id = pa.recurso_id
    )
)
INSERT INTO seguimiento_acceso
(fecha_yhora_entrada, fecha_yhora_salida, host, persona_id, recurso_id)
SELECT TIMESTAMP '2023-05-01 00:00:00'
      + ((rn % 30) * INTERVAL '1 day')
      + ((rn % 86400) * INTERVAL '1 second'),
TIMESTAMP '2023-05-01 00:20:00'
      + ((rn % 30) * INTERVAL '1 day')
      + ((rn % 86400) * INTERVAL '1 second'),
```

```
'host-' || ((rn % 10) + 1),
  persona_id, recurso_id
FROM faltantes;
```

Lectura del script.

- CROSS JOIN construye el producto cartesiano $|persona| \times |recurso|$.
- row_number() genera un rn determinista que se usa como semilla para distribuir las fechas (un día por cada 30 pares) y los hosts.
- El NOT EXISTS hace la operación *idempotente*: re-ejecutar el script no duplica pares ya cargados.

Verificación: tras correr el script, count(*) de seguimiento_acceso debería igualar $|persona| \cdot |recurso|$.

6.3. c) Información estadística del catálogo

Antes de mirar el catálogo conviene refrescar:

```
ANALYZE persona;
ANALYZE recurso;
ANALYZE seguimiento_acceso;
```

Por tabla — estimaciones del planner

```
SELECT n.nspname AS esquema,
       c.relname AS tabla,
       c.reltuples::bigint AS filas_estimadas,
       c.relpages AS paginas
FROM pg_catalog.pg_class c
JOIN pg_catalog.pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
      AND c.relname IN ('persona', 'recurso', 'seguimiento_acceso');
```

Esto se corresponde directamente con n_r y b_r del Teórico 6.

Por tabla — actividad

```
SELECT schemaname, relname,
       seq_scan, idx_scan,
       n_tup_ins, n_tup_upd, n_tup_del,
       n_live_tup, n_dead_tup,
       last_analyze
FROM pg_stat_user_tables
WHERE relname IN ('persona', 'recurso', 'seguimiento_acceso');
```

Si idx_scan = 0 para seguimiento_acceso significa que las consultas hechas hasta ahora no encontraron un índice útil (se resolvieron todas con *Seq Scan*).

Por columna — pg_stats

```
SELECT schemaname, tablename, attname,
       null_frac, n_distinct, avg_width,
       correlation,
       most_common_vals, most_common_freqs,
```

```

        histogram_bounds
FROM pg_stats
WHERE tablename IN ('persona', 'recurso', 'seguimiento_acceso')
ORDER BY tablename, attname;

```

Qué mirar concretamente:

- `n_distinct` en `persona_id` dentro de `seguimiento_acceso`: debería ser cercano al número de personas.
- `histogram_bounds` de `fecha_yhora_entrada`: refleja el rango temporal generado por el script de carga.
- `correlation` cerca de 1 indica que el orden físico coincide con el orden lógico, y favorece *Index Scan*.

6.4. d) Filtrado por fecha: plan inicial y mejora

Consulta del enunciado.

```

SELECT * FROM seguimiento_acceso
WHERE fecha_yhora_entrada = TIMESTAMP '2023-05-06 00:00:00';

```

d.a) Plan inicial

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM seguimiento_acceso
WHERE fecha_yhora_entrada = TIMESTAMP '2023-05-06 00:00:00';

```

Sin índice sobre `fecha_yhora_entrada` el plan habitual es:

```

Seq Scan on seguimiento_acceso (cost=...)
  Filter: (fecha_yhora_entrada = '2023-05-06 00:00:00'::timestamp)

```

Es decir, A1 del Teórico 6: recorre todos los bloques y descarta los que no cumplen el predicado.

d.b) Mejora

Para forzar un plan distinto se crea un índice B+tree sobre el atributo filtrado y se actualizan las estadísticas:

```

CREATE INDEX IF NOT EXISTS idx_seguimiento_acceso_fecha_entrada
  ON seguimiento_acceso (fecha_yhora_entrada);

ANALYZE seguimiento_acceso;

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM seguimiento_acceso
WHERE fecha_yhora_entrada = TIMESTAMP '2023-05-06 00:00:00';

```

Si la selectividad es muy alta (pocas filas con esa fecha exacta), el *planner* cambia a *Index Scan* o *Bitmap Index Scan*, que corresponde al algoritmo A5/A7 del Teórico 6 (selección por índice secundario).

```

Bitmap Heap Scan on seguimiento_acceso ...
  Recheck Cond: (fecha_yhora_entrada = '2023-05-06 00:00:00'::timestamp)
-> Bitmap Index Scan on idx_seguimiento_acceso_fecha_entrada

```

Observación. En el script de carga se usaron fechas *con hora distinta de 00:00:00*; por eso una igualdad estricta a '2023-05-06 00:00:00' puede devolver pocas o ninguna fila. Para “todo el día 2023-05-06” la consulta correcta es un rango:

```
SELECT * FROM seguimiento_acceso
WHERE fecha_hora_entrada >= DATE '2023-05-06'
AND fecha_hora_entrada < DATE '2023-05-07';
```

6.5. e) NATURAL JOIN: análisis y forzado de Merge Join

Consulta original.

```
SELECT * FROM seguimiento_acceso NATURAL JOIN persona;
```

e.a) Plan habitual — Hash Join

```
Hash Join (cost=...)
Hash Cond: (seguimiento_acceso.persona_id = persona.persona_id)
-> Seq Scan on seguimiento_acceso
-> Hash
    -> Seq Scan on persona
```

Lectura: PostgreSQL elige *hash join* porque, sin un orden previo sobre `persona_id` en ambas relaciones y sin un índice utilizable en línea, construir una tabla hash sobre la relación más chica (`persona`) y *probar* contra la grande (`seguimiento_acceso`) es lo más barato. Esto se corresponde con el algoritmo *Hash-Join* descrito en el Teórico 6.

e.b) Modificación para Merge Join

Para que el optimizador elija *Merge Join* la idea es darle entradas ordenadas por la clave de join. Hay dos formas complementarias:

1. Crear un índice ordenado sobre la columna del join en `seguimiento_acceso` (la otra ya tiene PK):

```
CREATE INDEX IF NOT EXISTS idx_seguimiento_acceso_persona_id
ON seguimiento_acceso (persona_id);
ANALYZE persona;
ANALYZE seguimiento_acceso;
```

2. Reescribir la consulta con join explícito y `ORDER BY` por la clave de join:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT sa.*, p.*
FROM seguimiento_acceso sa
JOIN persona p
ON sa.persona_id = p.persona_id
ORDER BY sa.persona_id;
```

Con esa información el planner suele elegir *Merge Join*:

```
Merge Join
Merge Cond: (sa.persona_id = p.persona_id)
-> Index Scan using idx_seguimiento_acceso_persona_id on seguimiento_acceso sa
-> Index Scan using persona_pkey on persona p
```

Verificación académica forzando el algoritmo

Si el motor sigue eligiendo Hash Join (por tamaños y costos estimados), se puede deshabilitar para esa sesión:

```
SET enable_hashjoin = off;
EXPLAIN (ANALYZE, BUFFERS)
SELECT sa.*, p.*
FROM seguimiento_acceso sa
JOIN persona p ON sa.persona_id = p.persona_id
ORDER BY sa.persona_id;
RESET enable_hashjoin;
```

Conclusión. El optimizador no usa fórmulas absolutas: compara costos estimados sobre las estadísticas del catálogo. Cuando uno crea índices adecuados o pide ordenamientos por la clave de join, está cambiando los *insumos* del cálculo, no las fórmulas. Eso es justamente lo que el Teórico 7 llama *optimización basada en costos*: el planner explora un espacio de planes equivalentes y elige el de menor costo estimado dado el estado actual de las estadísticas.

7. Conclusiones

El práctico recorre, en orden, los tres niveles donde se puede pensar la ejecución de una consulta:

1. **Algoritmos físicos** (Ejercicio 1): elegir A1...A11 en función de qué índices hay y de la forma del predicado.
2. **Reescritura algebraica** (Ejercicios 2 y 3): aplicar reglas de equivalencia para empujar selecciones y proyecciones hacia las hojas y reordenar joins para minimizar tamaños intermedios.
3. **Catálogo y planner reales** (Ejercicios 4 y 5): identificar en PostgreSQL dónde viven las estadísticas que el optimizador usa, y cómo se cambia el plan observado actuando sobre ellas (índices, `ANALYZE`, reescritura con `ORDER BY`, banderas como `enable_hashjoin`).

La idea común detrás de los cinco ejercicios es: *el plan de ejecución no es único*, y la diferencia de costos entre planes equivalentes puede ser de varios órdenes de magnitud; la tarea del DBA y del programador SQL es darle al motor estadísticas y estructuras que le permitan elegir bien.